

Deploying to production

Production is a single Oracle Cloud instance — **business2**, VM.Standard.A1.Flex (ARM64, 2 OCPU / 12 GB), region `us-phoenix-1`, public IP **129.146.22.240**.

Everything runs on that one box:

Container	What it is	Ports
<code>contracts-app</code>	the API, image <code>eradwan/contractsapp:1.0.1</code>	5100 (http), 8544 (https)
<code>erp-mysql-contracts</code>	the production database , volume <code>erp-mysql-volume-contracts</code>	3308 → 3306
<code>portainer</code>	container UI	9000, 8000

The three rules

1. **git pull before you build.** Building first gives a full Docker cache hit, the image tag never moves, and you deploy the previous build without noticing.
2. **Always build with `--cpuset-cpus 0`.** Compiling `Persistence.csproj` wants more than 6 GB and will pin both cores. The pin keeps core 1 free so SSH stays reachable.
3. **Never add `--memory` to the build.** A 6 GB cap makes Roslyn die with `System.OutOfMemoryException` mid-compile.

1 — On your machine

Only run the SPA build when there are frontend changes.

```
cd client-app && yarn build && cd ..  
git add -A && git commit -m "build DD-M" && git push
```

`vite.config.js` sets `emptyOutDir: true`, so Vite clears `API/wwwroot` itself — there is no need to delete it by hand, and one commit per deploy is enough.

Note: Vite loads **`vite.config.js`**, not `vite.config.ts` — `.js` comes first in Vite's resolution order and both files are committed. Keep any config change in the `.js` file, or delete `vite.config.js` + `vite.config.d.ts` so the `.ts` takes over (untested — the `.ts` enables `nodePolyfills` and `mermaid` handling that are currently inert).

2 — On the server

```
ssh ubuntu@129.146.22.240
tmux new -s build # the build survives a dropped connection

source ~/.telerik/telerik.env # Telerik NuGet key + license (see box below) — REQUIRED
cd ~/erp-contracts
git fetch origin && git reset --hard origin/master && git clean -fd
git log --oneline -1 # must match what you just pushed

DOCKER_BUILDKIT=0 sudo docker build -f Dockerfile.vm --cpuset-cpus 0 \
  --build-arg TELERIK_NUGET_KEY="$TELERIK_NUGET_KEY" \
  --build-arg TELERIK_LICENSE="$TELERIK_LICENSE" \
  -t eradwan/contractsapp:1.0.1 . 2>&1 | tee ~/diag/build.log

sudo docker images eradwan/contractsapp # the image ID MUST differ from the last deploy
docker compose -f docker-compose.vm.yml down
docker compose -f docker-compose.vm.yml up -d # same shell: compose reads $TELERIK_LICENSE for the runtime env
docker compose -f docker-compose.vm.yml ps
```

tmux: detach with **Ctrl-b** then **d**, come back with `tmux attach -t build`.

Telerik secrets (since the Sep 2026 Telerik Reporting upgrade)

`nuget.config` pulls `Telerik.*` from the private Telerik feed with `%TELERIK_NUGET_KEY%` as the password, and the app needs `TELERIK_LICENSE` (file *contents*) at build and run time or reports get a trial watermark. Keep both **outside the repo** — `git clean -fd` would delete a `.env` in `~/erp-contracts`. One-time setup on the server (the key is the `TELERIK_NUGET_KEY` value from your Mac shell / `nuget-keys` page — 113 chars, ends in `==`, nothing after it):

```
mkdir -p ~/.telerik && chmod 700 ~/.telerik
nano ~/.telerik/telerik-license.txt # paste your Mac's ~/.telerik/telerik-license.txt (one line)
printf '%s\n' \
  "export TELERIK_NUGET_KEY='PASTE_KEY_HERE'" \
  'export TELERIK_LICENSE="$(cat ~/.telerik/telerik-license.txt)'" \
  > ~/.telerik/telerik.env
chmod 600 ~/.telerik/telerik.env ~/.telerik/telerik-license.txt

source ~/.telerik/telerik.env
echo "key=${#TELERIK_NUGET_KEY} license=${#TELERIK_LICENSE}" # expect key=113 license=14248
```

`sudo` drops the environment, but `"$TELERIK_NUGET_KEY"` is expanded by your shell before `sudo` runs, so the values still reach the build. Re- `source` after every tmux reattach.

Restore errors and what they mean:

dotnet restore says	Cause	Fix
Value cannot be null or empty string. (Parameter 'password')	build args not passed / env empty	source the env file, check the lengths, rebuild
NU1301: Unable to load the service index for source https://nuget.telerik.com (API.csproj only)	key rejected (401) — typo, trailing char, or expired	curl -s -o /dev/null -w '%{http_code}\n' -u "api-key:\$TELERIK_NUGET_KEY" https://nuget.telerik.com/v3/index.json → 200 good, 401 regenerate
NU1101: Unable to find package Telerik.Licensing ... source(s): telerik	Telerik.Licensing lives on nuget.org, not the private feed	nuget.config maps it to nuget.org explicitly (fixed 2026-09-16); pull latest

Only API.csproj references Telerik packages, so the other four projects restoring fine while API fails is the signature of a feed/key problem, not a network one.

3 — Verify from outside

```
curl -s http://129.146.22.240:5100/ | grep -o 'index-[A-Za-z0-9_-]*\.js'
```

The SPA bundle filename fingerprints the deployed commit — match it against the bundle in your last commit. Do not skip this: a build can "succeed" and ship nothing (see rule 1).

To map any bundle name back to the commit that introduced it:

```
git log --diff-filter=A --oneline --name-only -- 'API/wwwroot/assets/index-*.js'
```

Shortcut: frontend-only releases

The SPA is committed to API/wwwroot and Dockerfile.vm has no npm step, so when a release touches only client-app/wwwroot the compiled backend is unchanged and no build is needed:

```
cd ~/erp-contracts && git pull
sudo docker cp API/wwwroot/. contracts-app:/app/wwwroot/
```

Serves instantly, no restart required. **Temporary** — it lives in the container, not the image, so the next `up -d` reverts it. Follow up with a real build when convenient.

Cautions

- **Never prune with --volumes.** erp-mysql-volume-contracts is the production database. `sudo docker image prune -f` and `sudo docker builder prune -f` are safe.
- **Use docker compose (v2, no hyphen).** The old `docker-compose` 1.29.2 is still installed and crashes with `KeyError: 'ContainerConfig'` when recreating a container — and it leaves the MySQL container **deleted**. If you must use it, run `down` before `up -d`.

- **Back up before risky work:** `bash sudo docker exec erp-mysql-contracts \ mysqldump -u root -p"$MYSQL_ROOT_PASSWORD_Contracts" --single-transaction --routines erp_contracts \ > ~/mysql-backups/contracts-golden/erp_contracts_$(date +%F_%H%M).sql`

If the box stops responding

Symptom: ports 22 and 9000 accept the TCP connection but nothing answers — SSH fails with `Connection timed out during banner exchange`. The kernel is alive; userspace is starved. (ICMP always fails here — OCI drops it. That is not a symptom.)

Cause is an unbounded on-box build: the compile needs >6 GB, and `docker build` runs inside `dockerd`, so closing your SSH session does **not** stop it. Mitigated now by a 4 GB swapfile (in `/etc/fstab`) and the `--cpuset-cpus 0` pin — but if it happens again:

1. OCI Console → Compute → Instances → **business2** → **Reboot**. If it sticks on "Stopping" for more than ~5 minutes, reboot again with the force option.
2. Restore service without building: `docker compose -f docker-compose.vm.yml up -d`.
3. Then investigate. A force reboot is a hard power cut to MySQL — take a dump once back in.

The instance has `restart: unless-stopped` on both services, so a reboot brings production back unattended.

Worth doing eventually

Stop building on the production host. The repo is public, so GitHub Actions' free ARM64 runners can build and push the image, leaving the server to `docker compose pull && up -d`. That removes this entire class of failure.